



SSA without Dominance for Higher-Order Programs

ROLAND LEISSA, University of Göttingen, Germany

JOHANNES MAX GRIEBLER, University of Göttingen, Germany

Dominance is a fundamental concept in compilers based on static single assignment (SSA) form. It underpins a wide range of analyses and transformations and defines a core property of SSA: every use must be dominated by its definition. We argue that this reliance on dominance has become increasingly problematic—both in terms of precision and applicability to modern higher-order languages. First, control flow overapproximates data flow, which makes dominance-based analyses inherently imprecise. Second, dominance is well-defined only for first-order control-flow graphs (CFGs). More critically, higher-order programs violate the assumptions underlying SSA and classic CFGs: without an explicit CFG, the very notion that all uses of a variable must be dominated by its definition loses meaning.

We propose an alternative foundation based on free variables. In this view, ϕ -functions and function parameters directly express data dependencies, enabling analyses traditionally built on dominance while improving precision and naturally extending to higher-order programs. We further present an efficient technique for maintaining free-variable sets in a mutable intermediate representation (IR). For analyses requiring additional structure, we introduce the *nesting tree*—a relaxed analogue of the dominator tree constructed from variable dependencies rather than control flow.

Our benchmarks demonstrate that the algorithms and data structures presented in this paper scale logarithmically with program size in practice.

CCS Concepts: • **Software and its engineering** → **Compilers**; • **Theory of computation** → *Program semantics*; *Data structures design and analysis*.

Additional Key Words and Phrases: static single assignment (SSA), dominance, higher-order intermediate representations, incremental free-variable analysis, persistent sets, nesting tree

ACM Reference Format:

Roland Leissa and Johannes Max Griebler. 2026. SSA without Dominance for Higher-Order Programs. *Proc. ACM Program. Lang.* 10, PLDI, Article 208 (June 2026), 24 pages. <https://doi.org/10.1145/3808286>

1 Introduction

Modern compilers rely on well-structured intermediate representations (IRs) to reason about programs. Traditional IRs in static single assignment (SSA) form express control flow explicitly via a control-flow graph (CFG) and use dominance to relate each use of a variable to its definition:

Property 1 (Dominance). Each use of a variable is dominated by its definition.

While this works well for first-order programs, dominance is inherently tied to CFGs. Dominance becomes imprecise or even undefined for higher-order programs. Yet higher-order functions are ubiquitous in modern code: in functional languages, in functional features of imperative languages like C++ and Java, and in data-parallel frameworks such as MapReduce or tensor computations.

Authors' Contact Information: [Roland Leissa](#), University of Göttingen, Göttingen, Germany, roland.leissa@cs.uni-goettingen.de; [Johannes Max Griebler](#), University of Göttingen, Göttingen, Germany, j.griebler@stud.uni-goettingen.de.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/6-ART208

<https://doi.org/10.1145/3808286>

λ -calculi handle higher-order programs naturally through block nesting: variables are explicitly scoped by their syntactic structure. However, this syntactic nesting can be too rigid for program transformations. For example, β -reducing the application $f\ a$ in the OCAML code in Fig. 1 duplicates g , although g does not depend on f 's variable x . For this reason, some IRs, such as THORIN [21], “CPS soup” in GUILLE [39], and MimIR [22], have abolished explicit scoping to simplify program transformations. However, these approaches currently lack a formal metatheory and efficient, principled techniques for organizing a higher-order “soup of functions”, in particular for reconstructing explicit scope nesting.

```
let f x =
  let g y = y in
  g x
```

Fig. 1. g does not depend on f .

This paper introduces λ_G , a graph-based λ -calculus for compiler IRs. Like basic blocks in SSA-based IRs, but unlike traditional λ -calculi, functions are “floating in a soup” and are *not* explicitly nested. Like traditional λ -calculi but unlike SSA-based IRs, λ_G naturally supports higher-order functions. Conceptually, λ_G replaces the CFG-based notion of dominance with a more relaxed relation of nesting: instead of asking “Is A dominated by B ?”, we ask “Is A nested within B ?”.

Contributions. This paper presents λ_G from two complementary perspectives—namely, as a *mathematical calculus* and as a *practical compiler IR* featuring efficient algorithms. In summary, this paper makes the following contributions:

- We present λ_G , a graph-based, typed λ -calculus that unifies functions and their bound variables through labels. It supports partially specified (unset) bodies and expresses (mutual) recursion through the graph structure rather than an explicit fixed-point operator. λ_G subsumes CFGs in SSA form and naturally extends to higher-order programs (Section 2).
- We introduce a novel framework for *incremental free-variable analysis* that *computes results lazily on demand* and *caches them* for subsequent queries. Compiler passes can freely mutate the program representation and query free variables at any time. The cache is *automatically and locally invalidated* when mutations occur, using a lightweight marking scheme that propagates invalidation only through affected parts of the program. This requires no manual reanalysis or synchronization with an external tool (Section 3.1).
- We formally define a *nesting relation* that specifies when a function must be nested within another. We prove the central insight of this work: nesting is a relaxed form of dominance. Unlike dominance, nesting depends solely on free variables rather than control flow, making it well-defined for higher-order programs. We restate the classic SSA-dominance Property 1 using nesting (Section 3.2).
- We propose a β -reduction algorithm for λ_G that specializes expressions only when necessary. Unlike conventional λ -calculi, our approach requires no block floating to hoist independent expressions out of functions. Furthermore, unlike scopeless representations such as CFGs, it avoids dominator trees and other control-flow preprocessing (Sections 3.3 and 3.4).
- We generalize critical edge elimination for higher-order functions through η -expansion. While η -expansion itself is standard, to our knowledge this connection between the two techniques has not been explored before (Section 3.5).
- We introduce the *nesting tree*, a relaxed alternative to the dominator tree that explicitly reconstructs the nesting structure of λ_G . Like nesting, it extends naturally to higher-order programs since its computation is independent of control flow. The nesting tree can optionally be enriched with information about loops and recursion (Section 4).
- We present an immutable data structure supporting efficient persistent set operations, enabling scalable maintenance of free-variable sets (Section 5).

- We implemented the concepts presented in this paper in MIMIR [22]. Our benchmarks demonstrate that the proposed algorithms and data structures scale log-linearly with program size on realistic workloads (Section 6).

All lemmas and theorems in this paper have been verified with the proof assistant LEAN.

2 Overview

This section first reviews classic SSA and CFGs. Then, we discuss λ_G and how to translate an SSA-based CFG to λ_G . Finally, we present core ideas that are explored in more detail in later sections.

2.1 CFG/SSA

As an introductory example, consider the nested loops in the SSA-form CFG in Fig. 3a. There is an outer loop with header h_i , exit block x_i , and body b_i that branches to the header h_j of the inner loop. This inner loop, in turn, exits via x_j and has a body b_j .

Suppose we want to peel or unroll the outer loop. To peel it, we must specialize $i1$ to $\emptyset$ (the first argument of $i1$'s ϕ -function); to unroll it, we must specialize $i1$ to $j2$ (the second argument of the ϕ -function). In both cases, specialization must apply not only to h_i but also to all basic blocks that depend on $i1$. Compilers such as LLVM typically use the dominator tree of the CFG (Fig. 3d) to identify these blocks, specializing all nodes dominated by the loop header. In this example, all nodes except f would be specialized. However, note that the inner loop is independent of $i1$. Therefore, it is sufficient to specialize only h_i , b_i , and x_i . This illustrates how control-flow information can overapproximate actual data dependencies (see also discussion of liveness in Section 7).

2.2 From SSA to λ_G

We now translate this program into λ_G (Fig. 3b). A λ_G program (Fig. 2) consists of *labels* mapped to *functions*. Each function is defined by a *type* and a *body*. The body is either an *expression* or *unset*, denoted by the symbol $\uparrow$. This enables partial or incremental graph construction. For instance, to construct the cyclic program in Fig. 3b, we must “tie the knot”. A straightforward approach is to first create all functions with unset bodies and then fill them in. We also allow the program graph to be mutated—either by introducing new label-to-function mappings or by modifying the body of an existing function.

Each function introduces a variable identified by the same label as the function itself. An expression may use ℓ to refer to the function and $\text{var } \ell$ to refer to its variable. The variable refers to the argument value bound upon application of the function, as in the standard λ -calculus. Since a program is a flat map from labels to functions, functions may reference variables defined in other functions *without explicit lexical scoping*, as in a CFG. For example, b_i uses the variable $i1 = \text{var } h_j$.

λ_G 's type system includes integers, booleans, tuple types, function types, and $\perp$ (bottom)—a type with no inhabitants. Functions whose codomain is $\perp$ are called *continuations*. Function application, tuple construction & extraction, and **let**-expressions—which allow for arbitrarily complex expressions—follow conventional syntax. λ_G also includes constants for integers and booleans, as well as built-in functions, some of which may be written in infix form for readability. In particular, we use a function br_T of type $[\text{bool}, [] \rightarrow T, [] \rightarrow T] \rightarrow T$ for a conditional branch yielding a value of type T . Thus, $\text{br}_\perp(\text{cond}, t, f)$ is like a conditional branch in LLVM or similar low-level IRs.

Using the correspondence between SSA and continuation-passing style (CPS) [2, 16], we translate all basic blocks in Fig. 3a with continuations in Fig. 3b. ϕ -Functions are represented as function variables, while their arguments become arguments to the application of the respective continuation, and the return point is made explicit: instead of an implicit return-terminator as in SSA, the

$P: \mathcal{L} \rightarrow f$
 $f := \lambda t \rightarrow t.b$
 $b := e \mid \uparrow$
 $e := c \mid \ell \mid \text{var } \ell \mid e e \mid (e, \dots, e) \mid e.i \mid \text{let } x = e; e \mid x$
 $t := \text{int} \mid \text{bool} \mid \perp \mid t \rightarrow t \mid [t, \dots, t]$

program: maps labels to functions
 function
 body: expr/unset
 expr: constant/function/var/app/tuple/extract/let
 type: int/bool/bottom/function type/tuple type

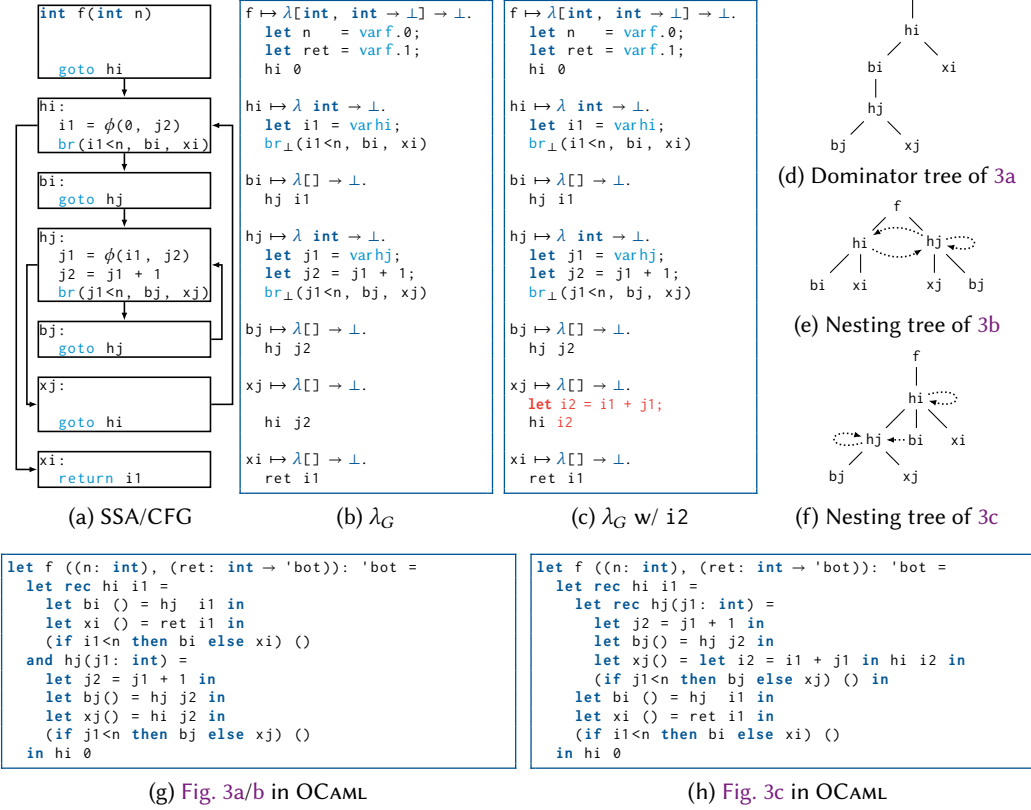
Fig. 2. Syntax of λ_G 

Fig. 3. Two nested loops. Inner loop (j1/j2) only depends on outer loop (i1) in 3c by i2.

Table 1. Computations and invalidation of free variables (m = mark)

run	f		hi		bi		hj		bj		xj		xi	
	m	FV	m	FV	m	FV	m	FV	m	FV	m	FV	m	FV
Fig. 3b	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	2	0	0	0	0	0	0	0	0	0	0	0	2	{var f, var hi}
	4	4	0	4 {var f}	4	{var f, var hi}	4	{var f}	4	{var hj}	4	{var hj}	2	{var f, var hi}
	5	5	0	5 {var f}	5	{var f, var hi}	5	{var f}	5	{var f, var hj}	5	{var f, var hj}	2	{var f, var hi}
	6	6	0	6 {var f}	6	{var f, var hi}	6	{var f}	6	{var f, var hj}	6	{var f, var hj}	2	{var f, var hi}
Fig. 3c	6	0	0	0	0	0	0	0	0	0	0	0	2	{var f, var hi}
	8	8	0	8 {var f}	8	{var f, var hi}	8	{var f, var hi}	8	{var hj}	8	{var hj, var hi}	2	{var f, var hi}
	9	9	0	9 {var f}	9	{var f, var hi}	9	{var f, var hi}	9	{var f, var hi, var hj}	9	{var f, var hi, var hj}	2	{var f, var hi}
	10	10	0	10 {var f}	10	{var f, var hi}	10	{var f, var hi}	10	{var f, var hi, var hj}	10	{var f, var hi, var hj}	2	{var f, var hi}

Table 2. SSA/CFG concepts and their analogues in λ_G

CFG w/ SSA	λ_G	CFG w/ SSA	λ_G
Function	Function	SSA dominance property	Well-Formedness
Basic block		Dominance	Free variables
Function parameter	λ -Variable	Dominator tree	Nesting tree
ϕ -Function		DJ-Graph	
Function call	Application	Call graph	w/ sibling dependencies
Jump		Loop Tree	
Function call argument	Application argument	Call graph SCCs	w/ SCCs
ϕ -Argument		Directly recursive function	Direct recursion
Function inlining		Natural loop	
Loop peeling	β -Reduction	Mutually recursive function	Mutual recursion
Loop unrolling		Irreducible loop	
CFG specialization		Critical edge elimination	η -Expansion

```

iter  $\mapsto \lambda[\text{int} \rightarrow \text{int}, \text{int}, \text{int}] \rightarrow \text{int}.$ 
  let f = variter.0;
  let n = variter.1;
  let x = variter.2;
  brrint(n <= 0, a, b)
a  $\mapsto \lambda[] \rightarrow \text{int}.$  x
b  $\mapsto \lambda[] \rightarrow \text{int}.$  iter (f, n - 1, f x)

```

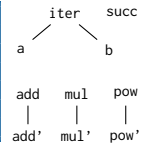
(a) iter computes $f^n(x)$

```

succ  $\mapsto \lambda \text{int} \rightarrow \text{int}.$  varsucc + 1
add  $\mapsto \lambda \text{int} \rightarrow \text{int} \rightarrow \text{int}.$  add'
add'  $\mapsto \lambda \text{int} \rightarrow \text{int}.$  iter (succ, varadd, varadd')
mul  $\mapsto \lambda \text{int} \rightarrow \text{int} \rightarrow \text{int}.$  mul'
mul'  $\mapsto \lambda \text{int} \rightarrow \text{int}.$  iter (add varmul, varmul', 0)
pow  $\mapsto \lambda \text{int} \rightarrow \text{int} \rightarrow \text{int}.$  pow'
pow'  $\mapsto \lambda \text{int} \rightarrow \text{int}.$  iter (mul varpow, varpow', 1)

```

(b) Successive use of iter to build power



(c) Nest. forest

Fig. 4. Higher-order function iter to construct the power function

continuation `ret` is passed to `f`, which `xi` invokes to exit `f`. This streamlining (treating basic blocks with ϕ -functions as functions) allows us to implement loop peeling/unrolling as β -reduction (inlining): We achieve peeling by β -reducing the call `hi 0` and unrolling by β -reducing `hi j2`.

2.3 Nesting Tree

As we have already argued, dominance only makes sense for first-order CFGs. So how shall we identify which functions depend on `hi` and need to be specialized as well? For this reason, we present the *nesting tree* depicted in Fig. 3e as an alternative to the dominator tree. We say “ ℓ_1 nests ℓ_2 ” if `var  $\ell_1$` occurs free in ℓ_2 (or transitively). Nesting implies dominance (Theorem 1) but not necessarily vice versa. The nesting tree is not only well-defined for higher-order programs—unlike the dominator tree—but is also more precise since it is constructed from the free variables of the functions (see Table 1 and Section 3.1). The nesting tree correctly captures that the inner loop is independent from the outer loop by placing them side by side. Thus, β -reducing `hi` will only specialize `hi`, `bi`, and `xi`. The nesting tree can be enhanced with *sibling dependencies* (the dotted arrows in Fig. 3e) that track how functions at the same level depend on each other (see Section 4.1.1).

Suppose we want to slightly change the program as indicated in the highlighted part in Fig. 3c. Note that applying this change to the SSA-form program in Fig. 3a does *not* change the dominator tree in Fig. 3d. However, the nesting tree *does* change as the inner loop now depends on the outer loop (see Fig. 3f). The nesting tree closely resembles the dominator tree, except that `hj` hangs directly under `hi`. The reason for this is that `bi` does not introduce any variables that `hj` depends on.

2.4 Higher-Order Functions and Direct Style

λ_G not only subsumes classical CFGs in SSA form, but also represents higher-order functions in both CPS and direct style. Figure 4a depicts a λ_G program that computes $f^n(x)$ in direct style. Figure 4b successively builds upon `iter` to construct a power function via currying and partial

application. For example, `pow 3 5` evaluates to 243. Figure 4c presents the resulting nesting forest of the program.

This example illustrates a qualitative advantage of λ_G over CFG-based IRs: λ_G can directly express higher-order functions like `iter`. CFGs cannot directly represent this program without further lowering. The nesting forest automatically recovers the dependency structure from free variables alone—without any explicit lexical scoping.

2.5 Summary

While the nesting tree is useful for making dependencies between functions explicit, *an explicit construction is rarely necessary*. In most cases, free-variable information suffices, as in the dependency-aware substitution described in Section 3.3.

This paper explores the relationship between SSA form and functional programming, and discusses further generalizations (summarized in Table 2) connecting SSA and λ_G . This relationship is not a strict mathematical isomorphism, but rather illustrates how key concepts in SSA correspond to those in λ_G . In particular, while free variables correspond to liveness in SSA (see Section 7), in Table 2 they serve as the primary structural notion in place of dominance.

3 Semantics

This section studies λ_G . Its most intricate feature is its scopeless graph structure, reminiscent of a CFG. We restrict our presentation to the rules for variables, functions, and applications, which capture the essential behavior of λ_G . For brevity, we omit rules for tuples and extractions as they are similar to applications, and we skip rules for constants since they are trivial. While λ_G includes **let**-bindings in the syntax, we omit them from the semantics. Adding them is straightforward but would introduce unnecessary notation without contributing new ideas. In practice, our implementation operates on a “Sea of Nodes” (SoN) (see Section 7), making explicit **let**-bindings unnecessary; they are used only in examples to provide a readable, textual representation of these graphs. One can think of these examples as if all **let**-bound variables had been inlined with their definitions, with the underlying graph representation sharing common (sub-)expressions as an implementation detail.

Typing. Since a λ_G program already consists of a label-to-function map P , we do not need an additional typing environment for type checking; instead, we look up labels directly in P (Rule T-FUN/T-VAR). Moreover, because function types are fully annotated, function bodies need not be type-checked at use sites (Rule T-FUN). This design allows the IR to compute and store the type of each expression at construction time. A function is type-checked when its body is defined (Rule T-BODY), and the entire program can be type-checked (Rule T-PROG) by verifying all function bodies and their subexpressions. The typing rules themselves are standard:

$$\begin{array}{c}
 \text{T-APP} \frac{P \vdash e_1 : t \rightarrow u \quad P \vdash e_2 : t}{P \vdash e_1 e_2 : u} \qquad \text{T-FUN} \frac{P(\ell) = \lambda t \rightarrow u. e}{P \vdash \ell : t \rightarrow u} \qquad \text{T-VAR} \frac{P(\ell) = \lambda t \rightarrow u. e}{P \vdash \text{var } \ell : t} \\
 \\
 \text{T-BODY} \frac{P \vdash e : u}{P \vdash \lambda t \rightarrow u. e} \qquad \text{T-PROG} \frac{P \vdash \lambda t_1 \rightarrow u_1. b_1 \quad \cdots \quad P \vdash \lambda t_n \rightarrow u_n. b_n}{\underbrace{\vdash \{ \ell_1 \mapsto \lambda t_1 \rightarrow u_1. b_1, \dots, \ell_n \mapsto \lambda t_n \rightarrow u_n. b_n \}}_P}
 \end{array}$$

3.1 Free Variables

We call the variables that occur in an expression up to (but not including) function expressions the *local variables* of the expression:

$$LV(\text{var } \ell) = \{\text{var } \ell\} \quad (1)$$

$$LV(e_1 e_2) = LV(e_1) \cup LV(e_2) \quad (2)$$

$$LV(\ell) = \emptyset \quad (3)$$

Similarly, we call the functions that occur in an expression up to (but not descending into) function expressions the *local functions* of the expression:

$$LF(\text{var } \ell) = \emptyset \quad (4)$$

$$LF(e_1 e_2) = LF(e_1) \cup LF(e_2) \quad (5)$$

$$LF(\ell) = \{\ell\} \quad (6)$$

This allows us to define *free variables* as the solution to the following recursive equations:

$$fv_P(\text{var } \ell) = \{\text{var } \ell\} \quad (7)$$

$$fv_P(e_1 e_2) = LV(e_1 e_2) \cup fv_P(LF(e_1 e_2)) \quad (8)$$

$$fv_P(\ell) = \emptyset \quad \text{if } P(\ell) = \lambda t \rightarrow u. \uparrow \quad (9)$$

$$fv_P(\ell) = fv_P(e) \setminus \{\text{var } \ell\} \quad \text{if } P(\ell) = \lambda t \rightarrow u. e \quad (10)$$

We obtain the final free-variable information as the least fixed point of the recurrence above:

$$FV_P(e) = \mu fv_P(e) \quad (11)$$

Implementation. Our implementation allows for efficient in-place mutation of function bodies, such as changing the body of x_j from Fig. 3b to 3c. Expressions, however, are immutable: once created, they cannot be modified. This allows us to store the local variables and functions during the construction of an expression, as this information remains sound regardless of subsequent mutations to the IR. Thus, local variables and functions can be obtained without descending into subexpressions.

Fixed-Point Iteration. If the compiler engineer queries the free variables of a function, we solve the fixed point and memoize the result so that subsequent queries return the memoized set directly. Expressions compute their free variables efficiently via Eq. (8), using local variables and the (memoized) free-variable sets of the local functions. To efficiently track which parts of the program require free-variable (re-)computation, and to prevent infinite cycles during the fixed-point iteration, we associate an integer mark with each function. Initially, mark is set to 0, indicating that the free-variable set is invalid. The fixed-point iteration increments a counter run in each iteration to distinguish the following cases:

- (1) mark = 0: The free-variable set is invalid; set mark to run and recompute it.
- (2) mark = run - 1: The free-variable set stems from the previous iteration. Set mark to run and add any additional variables discovered in this iteration.
- (3) mark = run: The expression belongs to the current iteration, indicating a cyclic dependency. In this case, we return the free-variable set accumulated so far.
- (4) Otherwise: The free-variable set is valid. Yield the memoized set without further computation. To distinguish cases 2 and 3, each new fixed-point iteration increments run by 2. After the fixed-point loop has terminated, no further traversals are required. The value of mark already indicates that the memoized free-variable set is valid (case 4).

To solve the data-flow equations, we require at most $d(G) + 2$ iterations, where $d(G)$ is the loop-connectedness of G . This is the maximum number of back edges on any acyclic path in G [10, 15].

A fixed-point loop requires two traversals for acyclic programs ($d(G) = 0$): the first computes the initial solution, and the second verifies that the fixed point has been reached. As described above, the fixed-point loop detects cyclic dependencies (case 3). This enables an additional optimization when no cycles occur. In this case, computation terminates immediately after the first iteration, as the computed free-variable sets are already sound.

Users & Invalidation. The memoized information remains valid as long as no referenced function is modified. For this reason, each function maintains a set of functions that *use* it:

$$UF_P(\ell) = \{\ell_{use} \mid \ell_{use} \rightarrow \ell\} \quad \text{where} \quad \text{Succ} \frac{P(\ell_1) = \lambda t \rightarrow u. e \quad \ell_2 \in LF(e)}{\ell_1 \rightarrow_P \ell_2} \quad (12)$$

If a function is modified, we transitively *invalidate* all functions in its *users* set by setting their mark to 0. This marks the memoized free-variable sets as invalid, requiring recomputation upon the next query. This requires traversing only a local part of the program: first, $fv_P(e)$ processes only expressions reachable from e ; second, expressions with valid free-variable sets require no further traversal; finally, computation is lazy and performed only on demand.

This user set is the *only* reverse dependency we require. In particular, we do not track classic def-use chains.

Example. Consider Fig. 3b and Table 1. Initially, mark is 0, indicating that the free-variable cache (column *FV*) is invalid. Suppose we wish to compute the free variables of x_i . These are given by the local variables of its body, united with the free variables of the local functions appearing in that body while removing the variable introduced in x_i itself:

$$FV_P(x_i) = LV(\text{ret } i1) \cup FV_P(LF(\text{ret } i1)) \setminus \{\text{var } x_i\} = \{\text{var } f, \text{var } hi\}$$

Local variables and local functions of every expression are precomputed at construction time, so no descent into subexpressions is required. Since this is a new computation, we increase *run* by two. Because the computation does not involve cycles, the result is already sound after a single iteration, and no further traversal is necessary.

Next, consider computing the free variables of f . We again increase *run* by two, to 4, and recursively compute free variables, marking each visited function with this value. Starting from f , we reach hi , then bi , hj , bj , and finally revisit hj , as indicated by $\text{mark} = \text{run} = 4$. This signals a cyclic dependency, and we therefore use the partial free-variable set for hj accumulated so far. Continuing with the computation for x_j requires revisiting hi , which is likewise marked with the current value of *run*; consequently, we again return the partial result computed so far. Finally, we reach x_i , whose mark indicates that its cached free-variable set is already sound, so we return the memoized set directly. This completes the first iteration of the fixed-point computation. In the second iteration ($\text{run} = 5$), we propagate $\text{var } f$ to bj and x_j . The computation stabilizes in the third iteration ($\text{run} = 6$), after which all computed *FV* sets are sound. Subsequent free-variable queries for any of these functions return the memoized results directly.

Now suppose we unset x_j , which invalidates all functions whose free-variable information depends on x_j , leaving only x_i valid. We then set the body of x_j to $hi \ i2$ (as in Fig. 3c) and again query the free variables of f . We increase *run* by two, to 8, and recursively compute free variables while updating *mark* accordingly. As before, x_i does not need to be processed, since its cached information remains valid. In contrast to Fig. 3b, $\text{var } hi$ now appears free in x_j in Fig. 3c, and is propagated during the fixed-point iteration. The computed sets are already sound in the second iteration ($\text{run} = 9$), which is confirmed by a third iteration ($\text{run} = 10$). See Section 5 for an efficient representation of these sets and their operations.

3.2 Nesting & Well-Formedness

Ultimately, we must determine the nesting structure induced by a λ_G program. Consider the program in Fig. 5: As `var f` appears free in `g`, `g` must be nested within `f`. As `var g` appears free in `h`, `h` must be nested within `g`. However, note that `var f` does *not* appear free in `h`. Nevertheless, `h` must also be nested within `f`. Therefore, the nesting relation must be transitive. We say ℓ_1 *strictly nests* ℓ_2 if $\ell_1 \succ_P \ell_2$, and ℓ_1 *nests* ℓ_2 if $\ell_1 \succeq_P \ell_2$:

```
f ↦ λ int → int. g 23
g ↦ λ int → int. h var f
h ↦ λ int → int. bar var g
```

Fig. 5. transitive nesting

$$\begin{array}{c}
\text{N-STRICT} \frac{\text{var } \ell_1 \in FV_P(\ell_2)}{\ell_1 \succ_P \ell_2} \quad \text{N-TRANS} \frac{\ell_1 \succ_P \ell_2 \quad \ell_2 \succ_P \ell_3}{\ell_1 \succ_P \ell_3} \\
\\
\text{N} \frac{\ell_1 \succ_P \ell_2}{\ell_1 \succeq_P \ell_2} \quad \text{N-REFL} \frac{\ell_1 = \ell_2}{\ell_1 \succeq_P \ell_2}
\end{array}$$

One may construct nonsensical, cyclic nesting dependencies. For example, `f` must be nested within `g` in Fig. 6 as `g`'s variable appears free in `f`. Conversely, `g` must be nested within `f` as `f`'s variable appears free in `g`: We prohibit such contradictory programs by requiring the nesting relation to be acyclic, i.e., antisymmetric. We say the program is *well-formed*:

```
f ↦ λ int → int. var g
g ↦ λ int → int. var f
```

Fig. 6. Nonsensical nesting

$$\text{WF} \frac{\forall \ell_1, \ell_2 \in \text{dom}(P) : \ell_1 \succeq_P \ell_2 \succeq_P \ell_1 \Rightarrow \ell_1 = \ell_2}{\vdash_{\text{wf}} P}$$

CFG & Dominance. A node n *dominates* node m in a directed graph with a unique start node, if all paths from the start to m also go through n [28]. In order to overlay a CFG over a λ_G program, we pick a root ℓ_0 and only consider functions as CFG nodes N that ℓ_0 nests—much like in a traditional CFG where other top-level functions like `printf` do not appear in the CFG where they are called:

$$\text{CFG}(P, \ell_0) = \langle N, \rightsquigarrow \rangle \quad \text{where} \quad N = \{ \ell \in \text{dom}(P) \mid \ell_0 \succeq_P \ell \} \quad \text{CFG-Succ} \frac{\ell_1 \rightarrow_P \ell_2 \quad \ell_1, \ell_2 \in N}{\ell_1 \rightsquigarrow \ell_2}$$

If the program is higher-order, the interpretation of the CFG becomes unclear. Nonetheless, we introduce this concept solely to show that nesting implies dominance. Reconsider Eq. (10): this equation is the sole location where a free variable is removed. In other words, a free variable propagates through function expressions unless it refers to its defining function:

Lemma 1 (One-Step Propagation). *If $\ell_1 \rightarrow_P \ell_2$ and $\text{var } \ell \in FV_P(\ell_2)$ and $\ell \neq \ell_1$, then $\text{var } \ell \in FV_P(\ell_1)$.*

PROOF. By definition of FV . □

This also holds for paths:

Lemma 2 (Path Propagation). *If $\ell_0 \rightarrow_P \dots \rightarrow_P \ell_k$ and $\text{var } \ell \in FV_P(\ell_k)$ and $\ell \notin \{\ell_0, \dots, \ell_k\}$, then $\text{var } \ell \in FV_P(\ell_0)$.*

PROOF. By induction on k and Lemma 1. □

This insight lets us prove that nesting implies dominance:

Theorem 1 (Nesting-Dominance). *If $\vdash_{\text{wf}} P$ and $\ell_0 \succeq_P \ell_1 \succeq_P \ell_2$, then ℓ_1 dominates ℓ_2 in $\text{CFG}(P, \ell_0)$.*

PROOF. Assume $\ell_0 \neq \ell_1 \neq \ell_2$. Otherwise the proof follows from basic properties of dominance. If $\text{var } \ell_1 \in FV_P(\ell_2)$ and there exists a path $\ell_0 \rightsquigarrow \dots \rightsquigarrow \ell_2$ that does *not* contain ℓ_1 , we obtain $\text{var } \ell_1 \in FV_P(\ell_0)$ by Lemma 2 and, hence, $\ell_1 \succeq_P \ell_0$. But this contradicts well-formedness, since we

also have $\ell_0 \succeq_P \ell_1$. The case where $\ell_1 \succ_P \ell_i \succ_P \ell_2$ then follows by induction and the transitivity of dominance. $\square$

Remark. The reverse does not necessarily hold. For example, *bi* dominates *hj* in Fig. 3d but *bi* does not nest *hj* in Fig. 3e. Additionally, note that ℓ_0 may have free variables.

Intuitively, well-formedness ensures that every variable use is properly “enclosed” by its definition—analogue to Property 1. However, our formulation does not rely on an explicit CFG or dominance relation. Instead, it uses the nesting relation, which naturally extends to higher-order programs where control flow is implicit. Accordingly, we introduce a structural property that paraphrases Rule WF and generalizes dominance beyond first-order control flow:

Property 2 (Nesting). Nesting must be antisymmetric.

3.3 Substitution

Substitution replaces free occurrences of a variable by an expression. In λ_G , substitution is more intricate than in the standard λ -calculus, since λ_G lacks explicit block nesting. Conversely, the structural nesting that does exist in a program may overapproximate the actual data dependencies between expressions—much like the dominator tree in a CFG. The substitution procedure defined in this section rewrites exactly those expressions that (transitively) depend on the substituted variable.

The central question in substitution is therefore whether a given expression e must be recursively rewritten or can be left unchanged. This decision depends on the free-variable set of e : if it contains the variable being substituted, e must be rewritten. As we traverse subexpressions, we may encounter functions whose bodies reference the substitution variable; in such cases, new functions with rewritten bodies are created. Any expression in which variables of these new functions occur free must also be updated accordingly. This mirrors the transitivity of the nesting relation (Rule N-TRANS).

To formalize this dependency-sensitive behavior, substitution must keep track of both variable and function rewrites. Specifically, we maintain a *variable map* V that maps variables to their replacement expressions, and a *function map* F that maps functions to their rewritten counterparts. Since substitution may introduce new label-to-function mappings, the program P must also be threaded through the process. Hence, substitution is defined as a function

$$\langle\langle V, F, P, e \rangle\rangle = \langle F', P', e' \rangle \quad (13)$$

which recursively rewrites an expression while threading the function and program maps. If the domain of V (the set of substitution variables) and the free variables of e are disjoint, no rewriting is required:

$$\langle\langle V, F, P, e \rangle\rangle = \langle F, P, e \rangle \quad \text{if } FV_P(e) \cap \text{dom}(V) = \emptyset \quad (14)$$

Otherwise, substitution proceeds recursively over all subexpressions:

$$\begin{aligned} \langle\langle V, F, P, e_1 e_2 \rangle\rangle &= \langle F'', P'', e'_1 e'_2 \rangle && \text{where } \langle\langle V, F, P, e_1 \rangle\rangle = \langle F', P', e'_1 \rangle && (15) \\ & && \text{and } \langle\langle V, F', P', e_2 \rangle\rangle = \langle F'', P'', e'_2 \rangle && \end{aligned}$$

If a free variable is encountered, its substitute is obtained directly from V . By Eq. (14), it must be present in V :

$$\langle\langle V, F, P, \text{var } \ell \rangle\rangle = \langle F, P, V(\text{var } \ell) \rangle \quad (16)$$

If a function expression is encountered, we either reuse its existing substitution

$$\langle\langle V, F, P, \ell \rangle\rangle = \langle F, P, F(\ell) \rangle \quad \text{if } \ell \in \text{dom}(F) \quad (17)$$

or create a new function with a rewritten body:

$$\begin{aligned} \langle\langle V, F, P, \ell \rangle\rangle &= \langle F', P'[\ell' \mapsto \lambda t \rightarrow u.e'], \ell' \rangle \quad \text{where } P(\ell) = \lambda t \rightarrow u.e, \ell' \text{ fresh,} \\ &\text{and } \langle\langle V[\text{var } \ell \mapsto \text{var } \ell'], F[\ell \mapsto \ell'], P[\ell' \mapsto \lambda t \rightarrow u.\uparrow], e \rangle\rangle = \langle F', P', e' \rangle \end{aligned} \quad (18)$$

In Eq. (18), we look up the function bound to ℓ in P , create a fresh label ℓ' mapped to a stub function with an unset body, and extend the variable and function maps so that occurrences of the old variable and function are redirected to their new counterparts. The unset body serves as a placeholder that resolves recursion: further rewrites will replace all occurrences of ℓ with ℓ' (see Eq. (17)). Once the rewritten body e' is obtained, it replaces the unset body of ℓ' .

Substitution preserves well-formedness and typing:

Lemma 3 (Substitution). *Let $\langle\langle \{\text{var } \ell \mapsto e_\ell\}, \emptyset, P, e \rangle\rangle = \langle F, P', e' \rangle$. If $\vdash_{\text{wf}} P$ and $\vdash P$ and $P \vdash e : t$ and $P \vdash e_\ell : t_\ell$, then $\vdash_{\text{wf}} P'$ and $\vdash P'$ and $P' \vdash e' : t$.*

PROOF. By induction on a derivation of $\vdash_{\text{wf}} P$, $\vdash P$, and $P \vdash e : t$. $\square$

3.4 β -Reduction

Building on substitution, we now define β -reduction

$$\beta \frac{P(\ell) = \lambda t \rightarrow u.e \quad \langle\langle \{\text{var } \ell \mapsto e_v\}, \emptyset, P, e \rangle\rangle = \langle F', P', e'_b \rangle}{P, \ell e_v \rightarrow_\beta P', e'_b}$$

and define full evaluation using a standard left-to-right, strict evaluation order:

$$\begin{array}{ccc} \text{V-FUN} \frac{}{\text{val } \ell} & \text{V-CONST} \frac{}{\text{val } c} & \text{E-APP1} \frac{P, e_1 \rightarrow P', e'_1}{P, e_1 e_2 \rightarrow P', e'_1 e'_2} \\ \\ \text{E-APP2} \frac{\text{val } e_1 \quad P, e_2 \rightarrow P', e'_2}{P, e_1 e_2 \rightarrow P', e'_1 e'_2} & & \text{E-}\beta \frac{\text{val } e_v \quad P, \ell e_v \rightarrow_\beta P', e'_b}{P, \ell e_v \rightarrow P', e'_b} \end{array}$$

Theorem 2 (Progress). *If $P \vdash e : t$ and $FV_P(e) = \emptyset$, then either $\text{val } e$ or $\exists P', e' : P, e \rightarrow P', e'$.*

Remark. Since e is closed, it follows from Lemma 2 that the part of the program P reachable from e is well-formed.

PROOF. By induction on a derivation of $P \vdash e : t$. $\square$

Theorem 3 (Preservation). *If $\vdash_{\text{wf}} P$ and $\vdash P$ and $P \vdash e : t$ and $P, e \rightarrow P', e'$, then $\vdash_{\text{wf}} P'$ and $\vdash P'$ and $P' \vdash e' : t$.*

PROOF. By induction on a derivation of $P, e \rightarrow P', e'$ and Lemma 3. $\square$

3.5 η -Conversion

Using the notion of free variables, we define η -reduction. Dually, for any expression of function type, we can perform η -expansion, wrapping it into an equivalent function expression:

$$\begin{array}{ccc} \eta\text{-RED} \frac{P(\ell) = \lambda t \rightarrow u.e \text{ var } \ell \quad \text{var } \ell \notin FV_P(e)}{P, \ell \rightarrow_\eta P, e} & & \eta\text{-EXP} \frac{P' = P[\ell \mapsto \lambda t \rightarrow u.e \text{ var } \ell] \quad \ell \text{ fresh} \quad P \vdash e : t \rightarrow u}{P, e \leftarrow_\eta P', \ell} \end{array}$$

Critical Edge Elimination. A *critical edge* in a CFG is an edge whose source block has multiple successors and whose destination block has multiple predecessors. Critical edges hinder many analyses and optimizations and are typically eliminated by inserting an empty intermediate block. For instance, removing `bi` in Fig. 3a would introduce a critical edge.

$\begin{array}{l} f \mapsto \lambda [\text{int}, \text{int}] \rightarrow \text{int}. \text{var } f.0 \\ g \mapsto \lambda ([\text{int}, \text{int}] \rightarrow \text{int}) \rightarrow \text{int}. B \\ h \mapsto \lambda [] \rightarrow \text{int}. \\ \quad f(x, y) + g f \end{array}$	$\begin{array}{l} f \mapsto \lambda [\text{int}, \text{int}] \rightarrow \text{int}. \text{var } f.0 \\ f_\eta \mapsto \lambda [\text{int}, \text{int}] \rightarrow \text{int}. f \text{ var } f_\eta \\ g \mapsto \lambda ([\text{int}, \text{int}] \rightarrow \text{int}) \rightarrow \text{int}. B \\ h \mapsto \lambda [] \rightarrow \text{int}. \\ \quad f(x, y) + g f_\eta \end{array}$	$\begin{array}{l} f' \mapsto \lambda \text{int} \rightarrow \text{int}. \text{var } f' \\ f_\eta \mapsto \lambda [\text{int}, \text{int}] \rightarrow \text{int}. f' (\text{var } f_\eta.0) \\ g \mapsto \lambda ([\text{int}, \text{int}] \rightarrow \text{int}) \rightarrow \text{int}. B \\ h \mapsto \lambda [] \rightarrow \text{int}. \\ \quad f' x + g f_\eta \end{array}$
---	---	--

(a) f used as callee and argument(b) f η -expanded to f_η (c) f now optimized to f' Fig. 7. η -Expansion makes *known* functions *well-known*; this allows us to adjust calls to the original function.

We now examine the corresponding situation in λ_G and remove bi in Fig. 3b. Eliminating bi yields the expression $br_\perp(i1 < n, hj, xi)$, which is ill-typed since hj has type $\text{int} \rightarrow \perp$: the argument $i1$ to hj is lost. Thus, when translating an SSA-form program to λ_G , all critical edges must be eliminated beforehand. Alternatively, one may use a branching function with a polymorphic type $\forall T. \forall U. [\text{bool}, T \rightarrow \perp, U \rightarrow \perp, T, U] \rightarrow \perp$ to pass ϕ -function arguments explicitly: $br(i1 < n, hj, xi, i1, ())$.

The key observation is that function hj is used *both* as a callee and as an argument to another function (br). We call a function expression *known* if it appears as the callee of an application, and *unknown* otherwise. A function is *well-known* if all of its function expressions are known [36]. Accordingly, hj is no longer well-known as it was in the original program before the critical edge was split via bi .

More generally, optimizations may wish to change a function's type—for example, to remove dead parameters. Consider Fig. 7a, where the second parameter of f is dead. We could create a specialized version f' that omits this parameter, but doing so would render $g f'$ ill-typed. However, if we first η -expand f to f_η and rewrite $g f$ as $g f_\eta$, then f becomes well-known (Fig. 7b). At that point, it is safe to remove the unused parameter (yielding f' , Fig. 7c) and to adjust all known function expressions, $f(x, y)$ to $f' x$ in this example.

In summary, if a function has both known and unknown function expressions, we can construct an η -expanded version and substitute it for all previously unknown occurrences. This transformation makes the original function well-known. For first-order programs that use only a branching function, this achieves precisely the effect of critical-edge elimination; for higher-order programs, it generalizes that concept.

3.6 Other Binders and Dependent Types

This framework naturally extends to dependent types, where types are themselves expressions: When gathering information, we must also traverse the corresponding types. For example, Eq. (2) is adjusted as follows:

$$LV(e_1 e_2) = LV(e_1) \cup LV(e_2) \cup LV(e_t) \quad \text{where } P \vdash e_1 e_2 : e_t \quad (19)$$

It is straightforward to support additional binders such as dependent pairs. The computations introduced above just have to be generalized to operate on arbitrary binders rather than functions alone. Our implementation in MIMIR already supports dependent types and several other binders. The other algorithms presented in this paper are extended accordingly to work on dependently typed expressions and/or other binders.

4 The Nesting Tree

While information about free variables is sufficient for many tasks within an IR, the *nesting tree* provides additional structural information. This data structure captures the minimal nesting required to reconstruct lexical scoping.

If the program is well-formed, we can represent the nesting relation as a forest of trees, where each tree is a relaxed dominator tree (Theorem 1). This is similar to CFGs, where each function has its own dominator tree. Analogous to the immediate dominator in a CFG, we introduce the concept

```

queue.push(make_node( $\ell_{root}$ ,  $\perp$ ))
vars ← {var  $\ell_{root}$ }
while not queue.empty():
    n ← queue.pop()
    for  $\ell \in \text{LF}(n.\text{function}.\text{body})$ :
        if not already_constructed( $\ell$ ):
            for (parent ← n; parent ≠  $\perp$ ; parent ← inest(parent)):
                if var  $\ell_{parent} \in \text{FV}(\ell)$ :
                    vars ← vars ∪ {var  $\ell$ }
                    queue.push(make_node( $\ell$ , parent))
                    break

```

Fig. 8. Nesting tree construction

of an *immediate nester*:

$$\text{inest}_P(\ell) = \begin{cases} \ell_{imm} & \text{if } \ell_{imm} \succ_P \ell \text{ and } \nexists \ell' : \ell_{imm} \succ_P \ell' \succ_P \ell \\ \perp & \text{if no such } \ell_{imm} \text{ exists } (\ell \text{ is root}) \end{cases} \quad (20)$$

To construct the nesting tree (Fig. 8), we traverse all functions reachable from a chosen root ℓ_{root} , following the nesting relation. For each newly discovered function ℓ , we attempt to place it as deep as possible: starting from the current node n , we check whether this node’s variable is free in ℓ . If not, we move up the partially constructed tree and repeat until we find the deepest ancestor whose variable is free in ℓ . We then attach ℓ as a child of that ancestor. Such an ancestor always exists because $\text{var } \ell_{root}$ is free in all reachable functions.

4.1 Case Study: Translation from λ_G to a Lexically Scoped Language

The nesting tree enables more structured analyses, which are best understood through a concrete example. We therefore describe how to translate a λ_G program into a functional language with explicit lexical scoping, such as OCAML. We consider two running examples: program A (Fig. 3b/g/e) and program B (Fig. 3c/h/f). Intuitively, the nesting tree determines how the “soup of functions” in λ_G must be arranged into lexical scopes so that free variables can access their definitions. For example, in programs A and B, function hi refers to $n = \text{var } f. \emptyset$. Consequently, hi must be nested within f in the OCAML translation.

4.1.1 Sibling Dependencies. We must also determine the order in which functions at the same nesting level are defined. For example, since bi invokes hj in program B, hj must be defined before bi . To formalize this, we introduce *sibling dependencies*, which capture how functions at the same level depend on one another. These dependencies induce an ordering on functions that share the same immediate nester. Formally, there is a sibling dependency from ℓ_1 to ℓ_2 if some function ℓ'_1 nested within ℓ_1 uses ℓ_2 , and ℓ_1 and ℓ_2 have the same immediate nester:

$$\text{SIBL} \frac{\ell_1 \succeq_P \ell'_1 \quad \ell'_1 \rightarrow_P \ell_2 \quad \text{inest}_P(\ell_1) = \text{inest}_P(\ell_2)}{\ell_1 \twoheadrightarrow_P \ell_2}$$

The continuation xi in programs A and B, for instance, does not depend on any of its siblings. It may therefore be placed at any position among them, provided it remains nested within hi .

4.1.2 Direct & Mutual Recursion. When sibling dependencies form cycles, they induce recursion. Acyclic sibling dependencies indicate that one subtree depends on another, but not vice versa. A self-loop denotes direct recursion, while longer cycles correspond to mutual recursion.

To identify these cycles, we run Tarjan’s strongly connected component (SCC) algorithm [37] *independently* on the sibling dependencies *at each level* of the nesting tree. This identifies hj as an inner loop of hi in program B. Only in the degenerate case of program A does the nesting tree merge both loops into a mutually recursive set of functions. As a useful byproduct, Tarjan’s algorithm also produces a topological ordering of the resulting SCCs. This enriched structure allows

us to traverse the nesting tree while tracking loop levels. In the OCAML translation, these cycles give rise to recursive bindings: a self-cycle produces a **let rec** definition, whereas longer cycles require **and**.

Natural loops are single-entry SCCs in a CFG: they have exactly one header node that *dominates* every node in the loop. *Irreducible loops* have multiple entry points (hence no single dominating header) and are therefore not natural loops. This reliance on dominance is a major reason why irreducible loops require special handling in classical CFG-based compilers. In the nesting tree, irreducible loops manifest as larger SCCs among sibling dependencies. Our analysis identifies them without special treatment, although subsequent loop optimizations may still require additional care.

We also introduce the concept of a *virtual root* that, by definition, nests every function. This makes it possible to construct a single nesting tree for the entire program, in which all closed functions become direct children of the virtual root. The sibling dependencies among these functions then induce a traditional call graph; our standard SCC detection directly identifies (mutual) recursion.

4.2 Case Study: Code Motion

As another use case, we consider code motion [7]. In a traditional SSA-based CFG, *early placement* places each expression as early as possible by first placing its subexpressions; the deepest of these placements in the dominator tree determines the earliest valid position of the expression itself. Conversely, *late placement* places expressions as late as possible by first placing all of their users; the least common ancestor of these placements in the dominator tree yields the latest valid placement.

Both placement strategies can be implemented directly using the nesting tree: for early placement, we consult an expression's level in the nesting tree; for late placement, we compute the least common ancestor of its users within the nesting tree.

In addition, the program may contain functions without a variable—such as `bi` in programs A and B—which are leaves of the nesting tree, often introduced during critical-edge elimination (Section 3.5). If we want to place expressions there as well, we must also consider sibling dependencies.

5 Efficient Set Operations

As discussed in Section 3.1, *each expression* stores the sets of local variables (*LV*) and local functions (*LF*). Furthermore, *each function* stores the set of functions that reference it (*UF*) as well as a cache of its free variables (*FV*). If this information were stored separately for each expression/function, the space requirements could easily explode. To avoid this blow-up, this section introduces a shared data structure that expressions reference rather than duplicate. Each expression/function therefore stores only a single pointer to its corresponding sets. As a result, copying reduces to a pointer copy, and checking for (in-)equality becomes a pointer comparison, which is required to determine whether the iteration described in Section 3.1 has reached a fixed point.

We first describe a naive implementation using hash-consed, ordered arrays, which performs well for small sets. Subsequently, we introduce an ordered trie with better space requirements. Finally, we index paths in this trie with a splay tree, as in a link-cut tree, to improve asymptotic runtime behavior. We consider the operations of insertion, union, removal, membership, and intersection testing. These are the fundamental set operations required to implement the algorithms discussed in this paper.

5.1 Hash-Consed, Ordered Arrays

This approach stores each set as an immutable, sorted array of unique elements. Each array is interned in a hash set, a technique known as *hash-consing*. Hash-consing ensures that identical

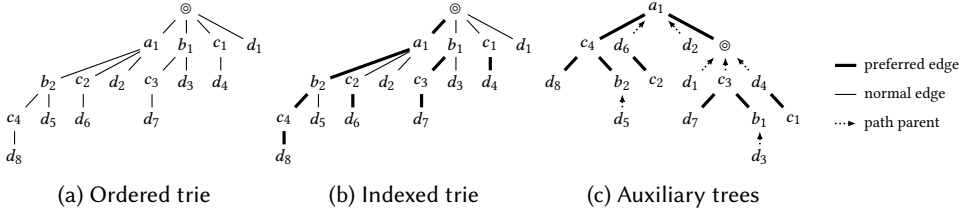


Fig. 9. Indexed trie: ordered trie where paths are indexed with auxiliary splay trees

arrays are stored only once globally, so equality and sharing reduce to constant-time pointer comparison.

Inserting an element into a set creates a new ordered array with the element placed in sorted order. *Removing* an element constructs a new array without that element. *Union* is computed via a linear-time merge of the two arrays, since the arrays are ordered. In each case, the resulting array is hash-consed, and the overall runtime scales linearly with the size of the set, plus the cost of hashing. *Membership* can be tested via binary search over the ordered array, yielding logarithmic-time complexity. However, as we use this data structure only for small sets, we found that a linear scan is slightly faster in practice. To test whether two sets intersect, we traverse the ordered arrays simultaneously, stopping as soon as a shared element is found. Other algorithms offer better average-case performance but come with higher constant overhead. Again, as we use this approach only for small sets, we opt for the simpler implementation.

This representation is simple and compact, with very low constant factors. We arena-allocate the arrays, which makes memory allocation cheap and array accesses more cache-friendly.

The abstract domain of the free variable analysis is the power set $\mathcal{P}(V)$ of the set V of $v := |V|$ program variables. If all elements of this domain were materialized, the total space required by the array representation would be

$$\sum_{i=0}^v i \binom{v}{i} = v 2^{v-1} = O(v 2^v). \quad (21)$$

In practice, however, the analysis constructs only those elements of $\mathcal{P}(V)$ that arise from analyzing program subexpressions. Since a program of size n has $O(n)$ subexpressions, and each subexpression gives rise to at most one new domain element, at most $O(n)$ distinct domain elements are materialized. As each such element contains at most n variables, the resulting space bound is $O(n^2)$. Likewise, we need at most $O(n^2)$ space to keep track of functions.

5.2 Ordered Trie

A more space-efficient alternative is a globally shared ordered trie that represents elements of the same abstract domain. Each element of $\mathcal{P}(V)$ corresponds to a unique node in a full trie, where the sequence of elements in the set is encoded by the path from that node to the root.

For example, the set $\{a, b, c, d\}$ is identified by the node labeled d_8 of the ordered trie in Fig. 9a. Walking up to the root $\odot$ yields c_4 , b_2 , and a_1 . The set $\{a, b, c\}$ is identified by the node labeled c_4 , reusing the same prefix nodes b_2 and a_1 .

Unlike the array-based representation, the trie shares common prefixes between sets. Consequently, if all elements of the abstract domain $\mathcal{P}(V)$ were materialized, the full trie would have exactly 2^v nodes. Under the same assumptions as above, however, only $O(n)$ domain elements arise during the analysis of a program of size n . In this case, the trie requires only $O(n)$ space, improving upon the quadratic bound of the array-based representation under the same assumptions. Likewise, we need at most $O(n)$ nodes in a trie to keep track of functions.


```

while not n1.is_root() and not n2.is_root():
    if n1.key < n2.key:
        # Search for n2.key in n1's splay structure
        n1 ← n1.find(n2.key) # Find n2.key or the element just greater than that
        if n1.key == n2.key:
            return True
        n1 ← n1.parent # Move upward
    else:
        # Symmetric case: search for n1.key in n2's splay structure
return False

```

Fig. 10. Intersection test for two sets organized in an indexed trie

However, although the sets are ordered, membership tests are linear in the depth of the trie because we do not have random access to the nodes along the path.

5.3 Indexed Trie

A *splay tree* [34] is a variant of a binary search tree that is self-adjusting. Instead of maintaining a fixed structure, it rotates an accessed node to the root through a process called *splaying*. This operation moves frequently accessed nodes closer to the root, improving access time for those elements over time.

A *link-cut tree* [33] is a data structure that supports dynamic trees, allowing efficient access to and updates of tree paths. It uses a series of splay trees to represent a forest of rooted trees, enabling operations such as path queries and tree merges to be performed in *amortized* $O(\log n)$ time.

Returning to the ordered trie, we *index* its paths using *auxiliary splay trees*, similar to a link-cut tree (see Figs. 9b to 9c). We never require the cut operation and only link nodes incrementally.

The trie is decomposed into *preferred paths* by assigning each node at most one *preferred child*, namely the child in the most recently accessed subtree. Preferred paths change dynamically with access patterns. Each preferred path is represented by an auxiliary splay tree whose nodes are keyed by depth in the trie. These auxiliary trees allow efficient navigation along paths by skipping large portions of them. To reconnect the auxiliary trees into the original trie structure, we use *path-parent* pointers. For each preferred path, exactly one node carries such a pointer: the node that is currently the root of the corresponding auxiliary splay tree. This node stores a pointer to its parent in the trie, which lies on a different preferred path.

This design reduces *membership test* complexity from linear to amortized logarithmic time. It also provides an opportunity to short-circuit insertions (removals), achieving logarithmic time whenever the element is already present (absent).

Intersection tests are required to implement Eq. (14). Although they remain linear in the worst case, the use of a link-cut tree enables several optimizations. First, we can detect shared prefixes in logarithmic time via a least common ancestor query. If one of the sets is small (16 elements or fewer—see Section 5.4), we perform at most 16 membership tests. Otherwise, we execute an alternating splay search: at each step, the current element of one set is located in the other via its splay structure, after which the roles are swapped. This symmetric strategy quickly discards large disjoint regions (see Fig. 10). This method achieves a runtime of $O(\log n + \log m)$ when the number of skipped elements dominates.

5.4 Implementation

Our implementation distinguishes four cases: empty sets are represented by a null pointer; singleton sets use a pointer that directly references the single element; small sets with up to 16 elements are stored as hash-consed, ordered arrays; and larger sets are represented using an indexed trie. Empirically, we found that this threshold (16 elements) marks the point at which the indexed trie outperforms the naive array-based implementation. We again use arena allocation for all trie nodes.

MIMIR maintains two global, immutable data structures: one for variables and one for functions. *All variable and function sets reference these data structures.* For example, all occurrences of the set $\{\text{var } f, \text{var } hi\}$ in Table 1 refer to the same memory location.

The ordering of trie nodes is arbitrary; therefore, we assign each node an increasing counter as its sort key. This design enables an additional optimization: nodes without a sort key have not yet been inserted into the trie and can thus be discarded quickly during operations such as membership tests. Moreover, each trie node records the minimum key along the path to the root, enabling further short-circuiting operations. Incrementing the counter only when necessary also helps keep the trie structure flat and efficient.

6 Evaluation

All the concepts introduced in this paper have been implemented in C++ as part of the MIMIR framework.

6.1 Experimental Setup

Benchmarks. To evaluate the scalability of our algorithms and data structures, we consider three synthetic benchmarks specifically constructed to stress-test their asymptotic behavior: (1) *Loop Cascade* consists of n consecutive counting loops, where each loop's bound depends on the final counter value of the previous loop. The loops are expressed in CPS (similar to Fig. 3) and embedded in a CPS function that provides the initial bound and a return continuation, which is invoked at the end with the last loop's counter. While not semantically meaningful, this benchmark captures the data-dependency patterns typical of sequential computations where each step depends on the result of the previous one. (2) *Accumulating Loop Cascade* extends this setup by summing all loop counters and passing the accumulated result to the return continuation. This modification keeps all loop counters live until the end, requiring their propagation as free variables from the final continuation back to their loop headers. This is the worst-case scenario for programs without loop nests: we introduce a number of variables proportional to the program size and keep them alive until the end. (3) *Loop Nest* is similar to *Loop Cascade*, but all loops are nested rather than consecutive. As a result, all loop counters must be propagated as free variables through every enclosing loop up to their defining headers. Thus, all variables must be propagated as in the *Acc. Loop Cascade*. This benchmark represents the worst case: it introduces a number of nested loops and variables proportional to the program size.

Algorithms. For each program, we measured the execution time of the following operations: (1) Computation of *free variables* for all functions in the generated program. (2) A β -reduction on the container function using a dummy value, producing a new function in which all occurrences of the original variable are replaced by the dummy. The free variables have already been computed in the previous step and are already cached. This operation particularly stresses the set intersection routines. (3) Construction of the *nesting tree*, including sibling dependencies, and SCCs.

Data Structures. To compare our custom indexed trie¹ used for tracking sets of variables and functions, we also implemented a reference implementation that uses `std::set` instead. In addition, we ported MIMIR to use the C++ library IMMER [29] (version 0.8.1). IMMER provides persistent, immutable data structures, including a persistent set type that we employed as an alternative to the indexed trie. It also supports transient (mutable) operations that allow efficient batch updates before reverting to a persistent form—a feature we use extensively, for example, during free-variable computations.

¹Including the optimizations discussed in Section 5.4; in particular, we use ordered, hash-consed arrays for small sets.

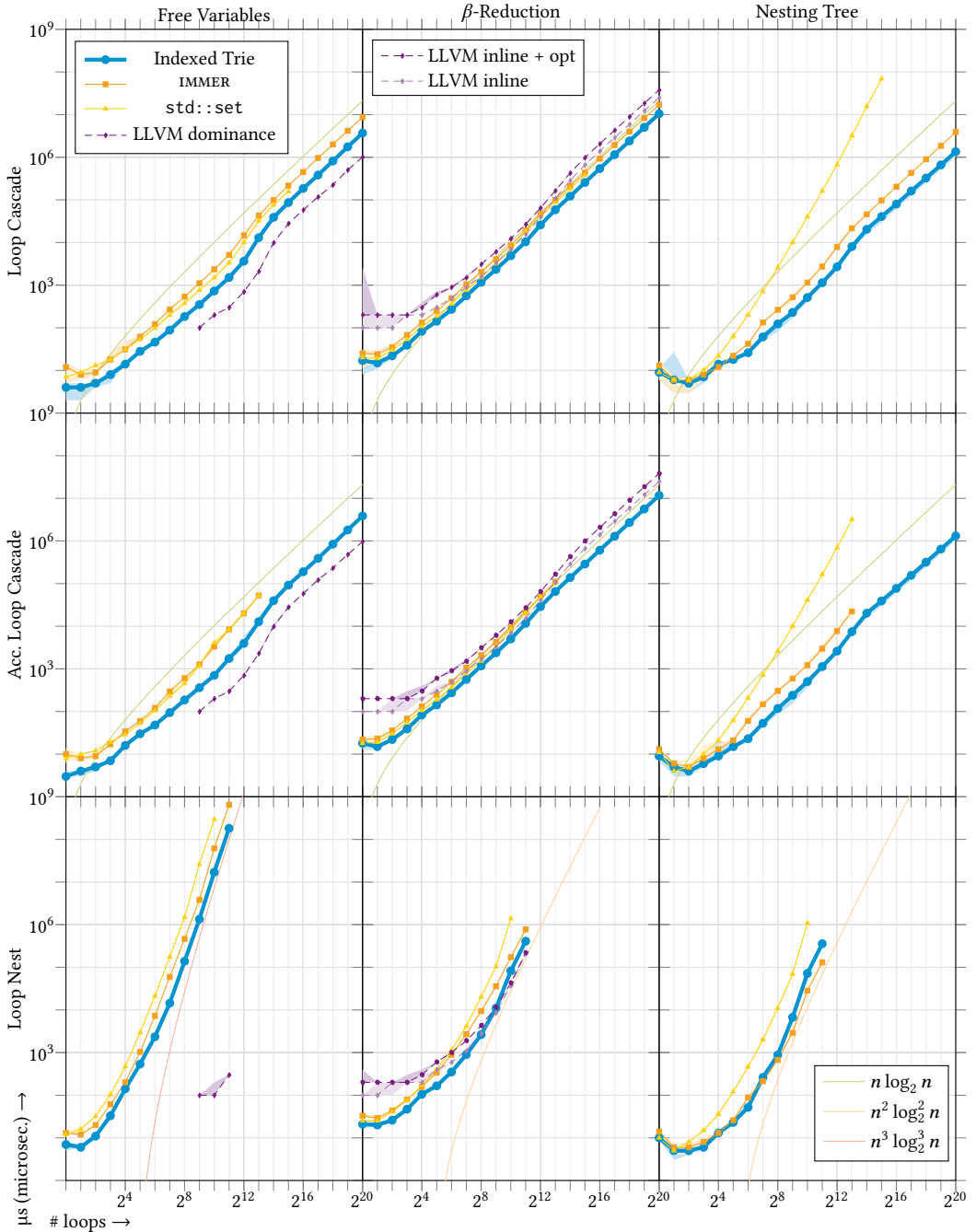


Fig. 11. Performance (in μs , lower is better) of three operations—free variable computation, β -reduction, and nesting tree computation—on three synthetic programs with increasing numbers of loops. The plots compare three different set implementations: the indexed trie, `IMMER`, and `std::set`. Additionally, free variable computation is compared against LLVM’s dominance analysis; β -reduction is compared against LLVM’s inliner (both with and without a sequence of `instcombine`, `early-cse`, `dce`, and `unreachableblockelim` to approximate MIMIR’s on-the-fly “Sea of Nodes” effects). For reference, $10^3 \mu\text{s} = 1 \text{ ms}$, $10^6 \mu\text{s} = 1 \text{ s}$.

Comparison with LLVM. We compiled all MIMIR benchmark programs to LLVM and compared LLVM’s dominance with MIMIR’s free-variable analysis, as well as LLVM’s inliner with MIMIR’s β -reduction. We emphasize that these comparisons are not apples-to-apples: MIMIR’s IR is strictly more expressive than LLVM’s, supporting first-class functions, polymorphism, dependent types, and plugin-defined abstract data types—which necessarily increases both IR size and traversal cost.

System Configuration. All experiments were conducted on a CachyOS system equipped with an AMD Ryzen AI 9 HX PRO 370 (12 physical cores, 24 logical processors, up to 5.16 GHz boost) and 32 GiB LPDDR5-7500 MT/s memory (effective clock ~ 3750 MHz). CPU frequency scaling was disabled (performance governor). All measurements were single-threaded. Each benchmark ran on an otherwise idle system, with processes pinned to a single core via taskset. All C++ code was compiled with GCC 15.2.1 using `-O3 -march=native -DNDEBUG`. Timings were measured via `std::chrono::steady_clock`. We measured LLVM using `opt` (version 22.1.1) by recording wall-clock time. These measurements are less precise than those obtained for MIMIR. Consequently, the lower end of the LLVM plots is either missing or appears noisy.

6.2 Discussion

Fig. 11 summarizes the results in double logarithmic scale. Each row corresponds to one benchmark strategy. The x-axis shows the number of generated loops, doubling at each tick. Each column corresponds to one of the measured operations. The y-axis reports the median execution time in microseconds (μ s) across nine runs, following a warm-up of three runs. The shaded regions indicate the observed range of measurements.

The indexed trie outperformed IMMER in most cases, except for nesting tree construction on *Loop Nest*, where IMMER performed better at larger input sizes. Often, the indexed trie was about twice as fast and, in some cases, around four times faster. Moreover, IMMER exhibited significantly higher memory consumption, causing out-of-memory termination of the *Acc. Loop Cascade* benchmark for programs exceeding 2^{13} loops. The `std::set` implementation roughly tracked IMMER’s performance in about half of the cases, but was significantly slower in the other half. Because `std::set` does not share any data, not only *Acc. Loop Cascade* but also *Loop Cascade* ran out of memory for input sizes exceeding 2^{13} loops. The indexed trie delivers the best overall performance; therefore, we focus on this data structure for the remainder of this discussion.

The measured MIMIR operations exhibit $n \log n$ scaling for both *Loop Cascade* and *Acc. Loop Cascade*. *Loop Nest* produces programs whose loop connectedness grows proportionally with program size: Each iteration of the fixed-point computation for free variables propagates through only a single nesting level, the indexed trie increases in depth, and nesting tree construction must traverse increasingly deep hierarchies to locate parent nodes. For smaller inputs, performance is dominated by hash-consed arrays; the asymptotic behavior becomes apparent only for larger instances. Empirically, the observed behavior at the high end is consistent with $O(n^3 \log^3 n)$ scaling for free-variable computation and $O(n^2 \log^2 n)$ scaling for β -reduction and nesting tree construction. To aid visual comparison, we include faint reference curves; these are shifted along the x-axis in the last row to align with the measurements. Due to this growth, we limited the *Loop Nest* benchmarks to 2^{11} loops. In this case, computing the free variables took ~ 3 min.

Deeply nested loops exceeding hundreds or thousands of levels are essentially absent in real-world software. For instance, within the entire SPEC2006 benchmark suite, the maximum observed loop-nesting depth is only 22 [see 26, Table 1]. We therefore conclude that the algorithms and data structures presented in this paper scale efficiently to programs with millions of data dependencies, except in contrived cases with unrealistically deep nesting.

LLVM’s dominance analysis is several times faster than MIMIR’s free-variable computation, as expected: LLVM uses a near linear algorithm that does not depend on the loop connectedness; for this reason, LLVM’s analysis remains efficient even for large *Loop Nests*. MIMIR’s free-variable computation, on the other hand, naturally extends to higher-order programs and is highly local and incremental, whereas LLVM must recompute the dominator tree on a per-function basis whenever the CFG changes. However, it is difficult to measure how well these advantages outweigh a much faster dominance analysis. MIMIR’s free-variable analysis can still be considered fast in absolute terms. For reference, computing free variables for more than three million functions comprising over one million (2^{20}) loops takes less than 4 s.

Despite operating on a richer IR, β -reduction in MIMIR is consistently faster than inlining in LLVM with the exception of large *Loop Nests*. Furthermore, MIMIR’s SoN representation gives rise to several optimizations: constant folding, arithmetic simplification, semi-global value numbering via hash-consing², as well as dead-code and unreachable-block elimination through graph traversal. To approximate these effects in LLVM, we additionally ran *instcombine*, *early-cse*, *dce*, and *unreachableblockelim* after inlining. Applying these optimizations further widens the gap.

To evaluate how MIMIR and LLVM compare on real-world programs with large control flow graphs, we used MIMIR’s *regex* plugin to compile a large regular expression matching a single HTTP access-log line, incorporating extensive enumerations of HTTP methods, status codes, timestamps, and path characters. The resulting matcher contains 1058 continuations in MIMIR and the same number of basic blocks when compiled to LLVM. Free-variable analysis took 476 μ s, while LLVM’s dominance analysis completed in ~ 100 μ s; β -reduction required 9755 μ s, whereas inlining took ~ 3800 μ s (~ 9400 μ s with optimizations). These numbers confirm that LLVM’s dominance analysis is several times faster than MIMIR’s free-variable analysis while β -reduction is competitive.

7 Related Work

Binders. λ_G identifies a function with the variable it introduces, so both share the same identifier. This removes the need to synchronize separate representations and avoids an entire class of book-keeping issues. Still, as Cockx [8] observes, “there are countless different techniques, frameworks, and meta-languages for dealing with name binding, none of which come close to be universally accepted or clearly superior to the others.” Because λ_G is a graph-based representation that supports mutation and (mutual) recursion via its cyclic structure rather than through an explicit fixed-point operator, many of the alternatives discussed by Cockx, such as (Co-)De Bruijn indices, are not suitable.

SSA Form. SSA form was introduced by Rosen et al. [31] and achieved widespread adoption in compiler design following the publication of an efficient construction algorithm by Cytron et al. [11]. **Property 1** is not stated as an explicit constraint; rather, it is guaranteed by that construction. Later work [e.g. 6, 14] formalized this guarantee as the *strict SSA property*. Without it, a use of a variable may appear in a block that is *not* dominated by its definition, causing standard analyses and transformations to break down. Consequently, modern IRs such as LLVM [19] and MLIR [20] treat dominance as integral to SSA well-formedness.

MLIR introduces a degree of higher-order behavior through regions that can be passed syntactically to operations specifically designed to consume regions. This complicates dominator-tree construction [4]. MLIR maintains a purely structural representation: regions resemble functions, may contain blocks, and can refer to free variables from enclosing scopes. Accordingly, MLIR embeds each region’s dominator tree into the parent CFG hierarchy. This may overapproximate data dependencies and place the region’s dominator tree unnecessarily deep. In contrast, our nesting tree

²Semi-global in the sense that expressions may float across function/basic-block boundaries while variables remain distinct.

computes the *minimal* required nesting purely from free variables, even in higher-order settings. Furthermore, MLIR is not a faithful higher-order representation, as it cannot directly represent the program in Fig. 4 without at least partially lowering the program.

Dominance in SSA. While efficient algorithms exist to compute the dominator tree [9, 25], the tree is highly brittle: any change to the CFG requires recomputation. Since the dominator tree is ubiquitous in SSA-based compilers, it must be frequently rebuilt whenever the CFG changes. This led compiler engineers to implement incremental updates to the dominator tree, avoiding full per-function recomputation [18]. Engineers must still manually track CFG mutations and keep the framework in sync. Our work sidesteps this entirely by asking how functions are *nested* and automatically maintaining free-variable sets in a transparent way, without manual intervention.

While this paper advocates free variables rather than dominance as the primary structural notion, dominance remains valuable in some contexts (e.g., classical SSA construction before SSA variables exist). There are also algorithms that construct SSA without dominance information [5, 24].

Explicit Scoping. Schneider’s verified compiler LVC [32] recovers SSA-style benefits in the lexically-scoped term language IL by replacing dominance with a syntactic *coherence* predicate that guarantees agreement between its imperative (IL/I) and functional (IL/F) semantics. Lemerre [24] “showed that we can also have both interpretations on a graph-based language without an explicit syntactic scope”. We are currently working on an SSA construction in MIMIR that extends Lemerre’s work for higher-order programs.

Kennedy [17] presents a CPS-based IR that represents the term structure as a doubly-linked tree, in which each subterm maintains an explicit up-link to its immediately enclosing parent. This design enables constant-time replacements, deletions, and insertions. Variable uses are maintained in a doubly-linked circular list, with same-binder equivalence classes computed via a union-find structure, enabling near-constant-time substitution when replacing one variable with another. However, the representation remains explicitly scoped and may still require block floating (see Section 1). Moreover, free-variable computation requires global re-analysis after every transformation. Given the destructive nature of the updates, there is no opportunity to precompute and incrementally maintain partial results as our framework does.

Sea of Nodes. A SoN-style IR [7] unifies data-flow and control-flow dependencies into a single graph-based representation, eliminating the rigid instruction ordering found in traditional CFG-oriented IRs. This approach has roots in the global value graph [30] and cyclic term graph [3]. It enables aggressive global optimizations by representing operations as nodes constrained only by their dependencies. Demange et al. [12] describe a formal semantics for SoN.

Soup of Functions. THORIN [21] pioneered a scopeless, higher-order CPS IR with primitive operations in direct style, while all expressions are managed in a SoN. MIMIR evolved from THORIN, retaining its higher-order SoN representation, and extends it to support direct-style functions, polymorphism, dependent types, and a plugin system [e.g. 38]. It also served as the implementation platform for the concepts presented in this paper. “CPS soup” [39] in GUILLE is a CPS-based IR that uses integers as labels for all continuations and maintains a flat, persistent map from labels to continuations, similar to a λ_G program. THORIN and MIMIR instead use the memory addresses of functions as labels. In contrast to GUILLE and THORIN, λ_G and MIMIR also support direct style. Furthermore, unlike MIMIR, GUILLE represents non-continuation expressions using traditional expression trees. THORIN, GUILLE, and MIMIR (prior to our work) overlay the soup of functions with a derived CFG (similar to Rule Succ) to compute properties such as dominance. This derived CFG, however, must be restricted to the *reachable and live* subset of the program, since neither unused continuations nor external callees (e.g., `printf`) should be treated as CFG nodes. For this reason,

these frameworks typically perform dedicated reachability and liveness analyses before applying β -reduction or similar transformations. Moreover, dominance still overapproximates the actual data dependencies, as discussed in Section 2. This paper advocates a paradigm shift by instead relying on free-variable information and avoiding a CFG overlay. In our setting, classical liveness corresponds exactly to free-variable information [1, p. 22]. Put another way, our framework tracks the liveness of variables, where each variable corresponds to the set of ϕ -functions introduced in the same basic block. Our new β -reduction algorithm, for example, visits only reachable functions without a prior global analysis, and uses free-variable information to decide locally which expressions must be specialized. We introduce a free-variable framework for “soup”-style IRs, develop a metatheory for this setting, and introduce nesting as a relaxed alternative to dominance.

Data Structures. We need an immutable data structure to keep track of sets of variables and functions. These data structures have a long history in functional programming and compiler implementation. By preserving previous versions under update, they enable persistent program representations that support efficient incremental queries and safe sharing across transformations [27]. Techniques such as path copying, hash-consing, and trie-based structures provide logarithmic or even amortized constant-time updates while avoiding destructive mutation. These properties make immutable data structures attractive for compilers and program analyses that require scalable maintenance of auxiliary information.

A link-cut tree [33] is a lesser-known data structure that proved highly useful in this work. Given the frequent occurrence of tree structures in compilers, we suspect that link-cut trees could be applied more broadly to accelerate various algorithms. For instance, they have already been used to speed up navigation of relational algebra trees in databases [13].

The sibling dependencies bear a strong resemblance to the join-edges in a DJ-graph [35]. In both cases, additional edges enrich a tree (the dominator tree or, in our case, the nesting tree) to capture non-hierarchical relationships. However, while the DJ-graph records control-flow joins, sibling dependencies capture data-flow relationships among siblings, making them applicable beyond first-order CFGs and suitable for higher-order or non-control-based IRs.

8 Conclusion

This paper revisits one of the central assumptions underlying SSA form: the reliance on dominance as the organizing principle for relating definitions and uses. While dominance has proven effective for first-order CFGs, it becomes increasingly imprecise and, in some cases, ill-defined in the presence of higher-order abstractions.

To address this, we introduced λ_G , a graph-based intermediate representation that replaces CFG-based dominance with nesting, a structural property derived from free-variable information. By treating free variables as the primary source of structural information, we extend classical SSA-style transformations, such as substitution and critical-edge elimination, to a higher-order, scopeless setting. Furthermore, we showed that this paradigm shift is practical: using an incremental caching framework and an indexed trie data structure, free-variable queries, dependency-aware β -reduction, and nesting-tree construction scale log-linearly for the benchmarks representative of real-world workloads.

More broadly, this work suggests that control-flow structure need not be the primary lens through which compiler analyses are expressed. By grounding program structure in data dependencies instead, we obtain a representation that more closely reflects the semantics of modern programming languages and enables optimizations and analyses that are difficult to express in traditional SSA frameworks.

Acknowledgments

The authors thank the anonymous reviewers for their helpful feedback, Joachim Meyer for implementing MimIR's regex plugin, Guido Moerkotte for insights on link-cut trees, and Sebastian Hack and Michel Steuwer for their support of the MimIR project.

Data-Availability Statement

The artifact accompanying this paper—including all benchmarks, measurement data, the LEAN mechanization of all lemmas and theorems, and the MimIR implementation used for the experiments—is publicly available on Zenodo [23]. The MimIR framework is developed at <https://mimir.github.io>, where all contributions from this paper have been integrated.

References

- [1] Andrew W. Appel. 1992. *Compiling with Continuations*. Cambridge University Press.
- [2] Andrew W. Appel. 1998. SSA is Functional Programming. *ACM SIGPLAN Notices* 33, 4 (1998), 17–20. doi:10.1145/278283.278285
- [3] Zena M. Ariola and Jan Willem Klop. 1996. Equational Term Graph Rewriting. *Fundam. Informaticae* 26, 3/4 (1996), 207–240. doi:10.3233/FI-1996-263401
- [4] Saurabh Bhat and Yuheng Niu. 2023. (Correctly) Extending Dominance to MLIR Regions. In *LLVM Developers' Meeting 2023*. San Diego, California, USA. <https://www.llvm.org/devmtg/2023-10/slides/techtalks/Bhat-Niu-ExtendingDominanceToMLIRRegions.pdf> Presentation slides.
- [5] Matthias Braun, Sebastian Buchwald, Sebastian Hack, Roland Leißa, Christoph Mallon, and Andreas Zwinkau. 2013. Simple and Efficient Construction of Static Single Assignment Form. In *Compiler Construction - 22nd International Conference, CC 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*. 102–122. doi:10.1007/978-3-642-37051-9_6
- [6] Preston Briggs, Keith D. Cooper, Timothy J. Harvey, and L. Taylor Simpson. 1998. Practical Improvements to the Construction and Destruction of Static Single Assignment Form. *Softw. Pract. Exp.* 28, 8 (1998), 859–881.
- [7] Cliff Click and Keith D. Cooper. 1995. Combining Analyses, Combining Optimizations. *ACM Trans. Program. Lang. Syst.* 17, 2 (1995), 181–196. doi:10.1145/201059.201061
- [8] Jesper Cockx. 2021. 1001 Representations of Syntax with Binding. <https://jesper.sikanda.be/posts/1001-syntax-representations.html>. Blog post, November 4, 2021.
- [9] Keith Cooper, Timothy Harvey, and Ken Kennedy. 2006. A Simple, Fast Dominance Algorithm. *Rice University, CS Technical Report 06-33870* (01 2006).
- [10] Keith D. Cooper, Timothy J. Harvey, and Ken Kennedy. 2004. *Iterative Data-flow Analysis, Revisited*. Technical Report. Department of Computer Science, Rice University. Available from Rice University.
- [11] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. 1991. Efficiently Computing Static Single Assignment Form and the Control Dependence Graph. *ACM Trans. Program. Lang. Syst.* 13, 4 (1991), 451–490. doi:10.1145/115372.115320
- [12] Delphine Demange, Yon Fernández de Retana, and David Pichardie. 2018. Semantic reasoning about the sea of nodes. In *Proceedings of the 27th International Conference on Compiler Construction, CC 2018, February 24-25, 2018, Vienna, Austria*, Christophe Dubach and Jingling Xue (Eds.). ACM, 163–173. doi:10.1145/3178372.3179503
- [13] Philipp Fent, Guido Moerkotte, and Thomas Neumann. 2023. Asymptotically Better Query Optimization Using Indexed Algebra. *Proc. VLDB Endow.* 16, 11 (2023), 3018–3030. doi:10.14778/3611479.3611505
- [14] Sebastian Hack, Daniel Grund, and Gerhard Goos. 2006. Register Allocation for Programs in SSA-Form. In *Compiler Construction, 15th International Conference, CC 2006, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2006, Vienna, Austria, March 30-31, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 3923)*, Alan Mycroft and Andreas Zeller (Eds.). Springer, 247–262. doi:10.1007/11688839_20
- [15] John B. Kam and Jeffrey D. Ullman. 1976. Global Data Flow Analysis and Iterative Algorithms. *J. ACM* 23, 1 (1976), 158–171. doi:10.1145/321921.321938
- [16] Richard Kelsey. 1995. A Correspondence between Continuation Passing Style and Static Single Assignment Form. In *Proceedings ACM SIGPLAN Workshop on Intermediate Representations (IR'95), San Francisco, CA, USA, January 22, 1995*. 13–23. doi:10.1145/202529.202532
- [17] Andrew Kennedy. 2007. Compiling with continuations, continued. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming, ICFP 2007, Freiburg, Germany, October 1-3, 2007*, Ralf Hinze and Norman Ramsey (Eds.). ACM, 177–190. doi:10.1145/1291151.1291179

- [18] Piotr Kuderski. 2017. Dominator Trees and incremental updates that transcend time. In *LLVM Developers' Meeting 2017*. San Jose, California, USA. https://llvm.org/devmtg/2017-10/slides/Kuderski-Dominator_Trees.pdf Presentation slides.
- [19] Chris Lattner and Vikram S. Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *2nd IEEE / ACM International Symposium on Code Generation and Optimization (CGO 2004)*, 20-24 March 2004, San Jose, CA, USA. IEEE Computer Society, 75–88. doi:10.1109/CGO.2004.1281665
- [20] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques A. Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2021, Seoul, South Korea, February 27 - March 3, 2021*, Jae W. Lee, Mary Lou Soffa, and Ayal Zaks (Eds.). IEEE, 2–14. doi:10.1109/CGO51591.2021.9370308
- [21] Roland Leiða, Marcel Köster, and Sebastian Hack. 2015. A graph-based higher-order intermediate representation. In *Proceedings of the 13th Annual IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2015, San Francisco, CA, USA, February 07 - 11, 2015*. 202–212. doi:10.1109/CGO.2015.7054200
- [22] Roland Leiða, Marcel Ullrich, Joachim Meyer, and Sebastian Hack. 2025. MimIR: An Extensible and Type-Safe Intermediate Representation for the DSL Age. *Proc. ACM Program. Lang.* 9, POPL (2025), 95–125. doi:10.1145/3704840
- [23] Roland Leiða and Johannes Griebler. 2026. SSA without Dominance for Higher-Order Programs. doi:10.5281/zenodo.19069679
- [24] Matthieu Lemerre. 2023. SSA Translation Is an Abstract Interpretation. *Proc. ACM Program. Lang.* 7, POPL (2023), 1895–1924. doi:10.1145/3571258
- [25] Thomas Lengauer and Robert Endre Tarjan. 1979. A Fast Algorithm for Finding Dominators in a Flowgraph. *ACM Trans. Program. Lang. Syst.* 1, 1 (1979), 121–141. doi:10.1145/357062.357071
- [26] Kai Nie, Qinglei Zhou, Hong Qian, Jianmin Pang, Jinlong Xu, and Xiyan Li. 2021. Loop Selection for Multilevel Nested Loops Using a Genetic Algorithm. *Mathematical Problems in Engineering* 2021 (2021), 1–18. doi:10.1155/2021/6643604
- [27] Chris Okasaki. 1999. *Purely functional data structures*. Cambridge University Press.
- [28] Reese T. Prosser. 1959. Applications of Boolean matrices to the analysis of flow diagrams. In *Papers Presented at the December 1-3, 1959, Eastern Joint IRE-AIEE-ACM Computer Conference* (Boston, Massachusetts) (IRE-AIEE-ACM '59 (Eastern)). Association for Computing Machinery, New York, NY, USA, 133–138. doi:10.1145/1460299.1460314
- [29] Juan Pedro Bolívar Puente. 2017. Persistence for the masses: RRB-vectors in a systems language. *Proc. ACM Program. Lang.* 1, ICFP (2017), 16:1–16:28. doi:10.1145/3110260
- [30] John H. Reif and Harry R. Lewis. 1986. Efficient Symbolic Analysis of Programs. *J. Comput. Syst. Sci.* 32, 3 (1986), 280–314. doi:10.1016/0022-0000(86)90031-0
- [31] B. K. Rosen, M. N. Wegman, and F. K. Zadeck. 1988. Global value numbers and redundant computations. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL '88). Association for Computing Machinery, New York, NY, USA, 12–27. doi:10.1145/73560.73562
- [32] Sigurd Schneider. 2018. *A verified compiler for a linear imperative / functional intermediate language*. Ph.D. Dissertation. Saarland University, Saarbrücken, Germany. <https://publikationen.sulb.uni-saarland.de/handle/20.500.11880/27296>
- [33] Daniel Dominic Sleator and Robert Endre Tarjan. 1983. A Data Structure for Dynamic Trees. *J. Comput. Syst. Sci.* 26, 3 (1983), 362–391. doi:10.1016/0022-0000(83)90006-5
- [34] Daniel Dominic Sleator and Robert Endre Tarjan. 1985. Self-Adjusting Binary Search Trees. *J. ACM* 32, 3 (1985), 652–686. doi:10.1145/3828.3835
- [35] Vugranam C. Sreedhar and Guang R. Gao. 1995. A Linear Time Algorithm for Placing phi-nodes. In *Conference Record of POPL'95: 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Francisco, California, USA, January 23-25, 1995*, Ron K. Cytron and Peter Lee (Eds.). ACM Press, 62–73. doi:10.1145/199448.199464
- [36] Guy L. Steele. 1978. *Rabbit: A Compiler for Scheme*. Technical Report. USA.
- [37] Robert Endre Tarjan. 1972. Depth-First Search and Linear Graph Algorithms. *SIAM J. Comput.* 1, 2 (1972), 146–160. doi:10.1137/0201010
- [38] Marcel Ullrich, Sebastian Hack, and Roland Leiða. 2025. MimIRaDe: Automatic Differentiation in MimIR. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction, CC 2025, Las Vegas, NV, USA, March 1-2, 2025*, Daniel Kluss, Sara Achour, and Jens Palsberg (Eds.). ACM, 70–80. doi:10.1145/3708493.3712685
- [39] Andy Wingo. 2023. *Approaching CPS Soup*. Igalia, S.L. / wingolog.org. <https://wingolog.org/archives/2023/05/20/approaching-cps-soup> Accessed: 2026-02-25.

Received 2025-11-14; accepted 2026-04-03